

Examen Final de GRAU-IA

(14 de enero de 2026)

Duración: 2 horas 30 minutos

1. (5 puntos) Una gran operadora de telecomunicaciones (telefonía, televisión a la carta e internet) quiere recomendar nuevos productos personalizados para sus clientes, ya sean de 1) telefonía móvil (compuesto por los servicios de telefonía, internet móvil y mensajería SMS/MMS), 2) internet+TV por cable (compuesto por internet por cable y televisión a la carta), o 3) ambos. Para ello quiere construir un sistema que recoge un montón de información del usuario (que ya tienen en sus sistemas informáticos):

- sobre su uso del móvil: se guarda toda la información relevante de las llamadas (destinatario, fecha, hora, duración) para poder extraer información como por ejemplo cuantas llamadas hace en horario de mañana, tarde y noche, cuantas llamadas hace en días laborables y cuantas en fin de semana, a cuantos operadores diferentes son sus llamadas, y cual es más o menos la duración de cada tipo de llamada. Del destinatario de la llamada se guarda su número de teléfono y la operadora de telecomunicaciones a la que pertenece. También se guarda la información sobre si usa internet en el móvil, y en caso positivo el número de GB de datos que en media suele consumir al mes. También se guarda información sobre cuantos mensajes SMS/MMS envía al mes. Además se guarda información sobre la tarifa actual de telefonía móvil que tiene el cliente (nombre comercial de la tarifa, precio mensual, precio de consumo mínimo, precio de establecimiento de llamada, número de minutos incluidos, velocidad de conexión de datos, precio por SMS, precio por MMS, los meses de permanencia) y también de las tarifas anteriores que haya tenido con esta operadora en los últimos 10 años.
- sobre su uso de internet por cable: el número de meses que ha contratado el internet con esta operadora (solo cuentan los últimos 10 años), el número de GB de datos de bajada que en media suele consumir al mes, el número de GB de datos de subida que en media suele consumir al mes, si requiere una IP fija, si dispone de modem wifi, si ha contratado servicio de firewall, si ha contratado servicio de asistencia telefónica.
- sobre su uso de la televisión por cable: se guarda toda la información sobre los canales que puede ver (de cada uno se tiene nombre y número de canal), los programas que se ofrecen en cada canal, y los que finalmente el usuario ha visto (para cada programa se guarda un sumatorio de los minutos que ha visto). De todos los programas se tiene también el nombre, la duración media (en minutos), la clasificación por edades (apto, mayores de 3, mayores de 7, mayores de 12, mayores de 18), el año de la primera emisión y si se ha de pagar aparte para verlo (pay-per-view). Los programas se clasifican a su vez en informativos, documentales, culturales, entretenimiento, deportes, corazón, series de ficción, infantiles, películas y musicales. En el caso de programas del tipo serie de ficción se guarda también el número de temporada y el número de episodio. En el caso de las películas se guarda el nombre del director y el género (cine clásico, animación, ciencia ficción, comedia, drama, acción, etc.)

A partir de toda esta información se puede elaborar un perfil de las características del usuario basándonos en un conjunto de atributos: fidelidad del usuario (alta si lleva más de 3 años contratando nuestros productos, baja en caso contrario), uso de teléfono (bajo, moderado, alto, intensivo), destino de llamadas (al operador, a varios operadores), uso de mensajería (bajo, medio, alto), uso de internet móvil (nulo, bajo, medio, intensivo), uso de internet fijo (nulo, bajo, medio, intensivo), uso de televisión (nulo, ocasional, normal, elevado) y perfil de visionado (ninguno, variado, mayormente informativo, mayormente cultural, mayormente ficción, mayormente infantil, mayormente corazón).

Las diferentes tarifas de telefonía móvil a recomendar también se pueden describir mediante una serie de características que nos permitirán identificar mejor que conjunto de tarifas son las más adecuadas para el usuario. Una tarifa puede ser descrita por su cuota mensual (ninguna, baja, media, alta), la exigencia de un consumo mínimo (no, bajo, medio, alto), el coste de establecimiento de llamada (ninguno, bajo, medio, alto), minutos de llamada incluidos en la cuota mensual (medio, alto, muy alto), velocidad de internet móvil (no dispone, 3G, 4G, 5G), los GB de datos incluidos en la tarifa (ninguno, menos de 2 GB, entre 3 y 4 GB, más de 4 GB), coste de los mensajes (bajo, medio, alto), exigencia de permanencia (alta, media, baja).

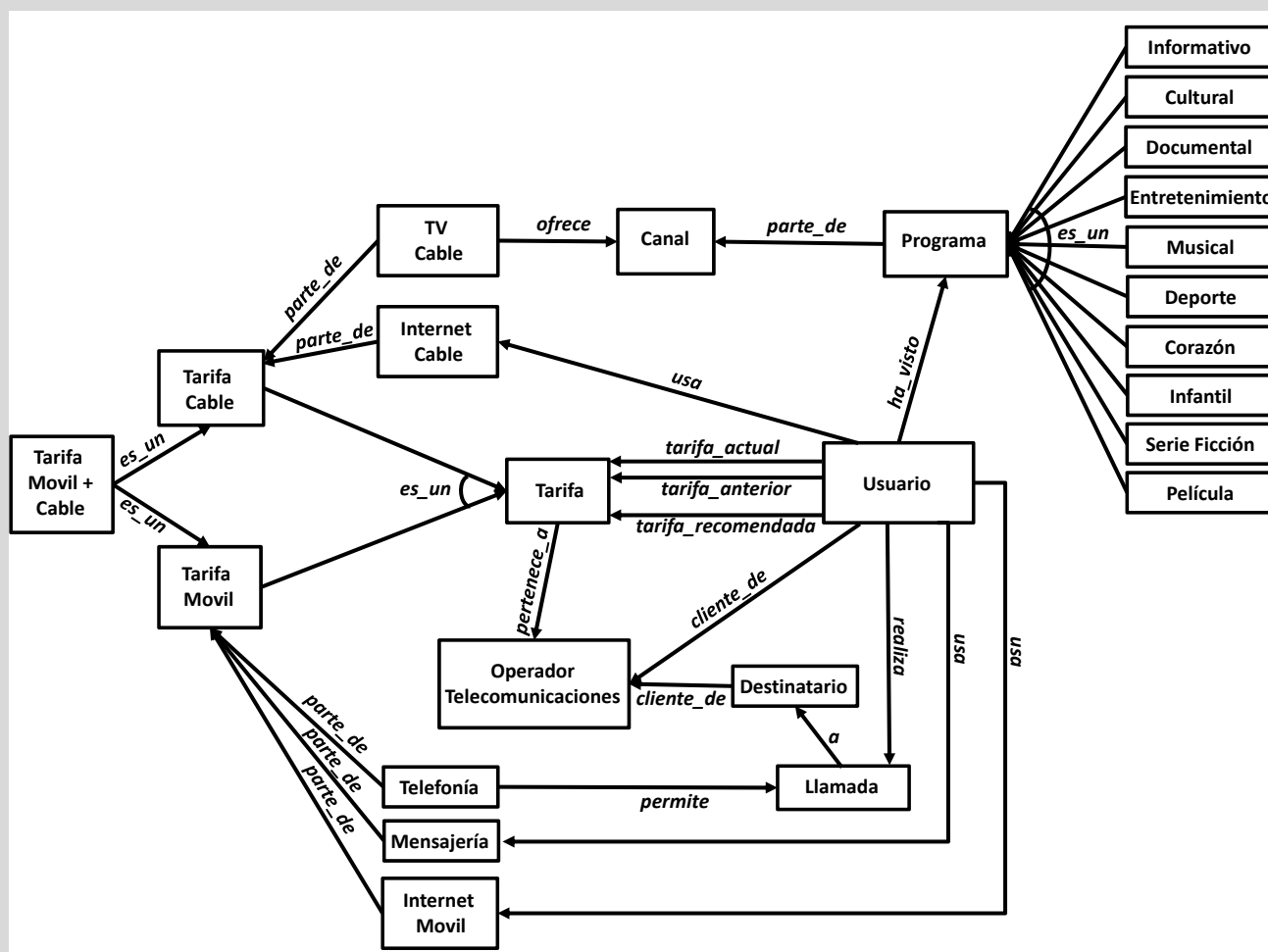
Las diferentes tarifas de internet+TV por cable también se pueden describir por una serie de características: cuota mensual (baja, media, alta), velocidad de bajada de datos (básica, media, alta), la velocidad de subida de datos (básica, media, alta), oferta de canales (básica, ampliada, máxima), target TV (infantil, juvenil, toda la familia, informativo, cultural, deportes, corazón).

A partir del perfil del usuario podemos asociar los valores de las características de las tarifas de telefonía e internet+TV más adecuada. Por ejemplo, si la fidelidad del usuario es baja y hace un uso alto de telefonía móvil, puede convenirle una tarifa que tenga una exigencia de consumo mínimo alta y que tenga una cuota mensual media. Si la fidelidad del usuario es alta y su uso de teléfono es bajo podrá tener una tarifa telefónica con una cuota mensual baja y con una exigencia de permanencia alta. Si el uso del teléfono es intensivo necesitará un coste de establecimiento de llamada bajo, sin consumo mínimo y con los minutos de llamada incluidos en la tarifa altos o muy altos. Si el uso de internet fijo es intensivo y el uso de televisión es nulo entonces le recomendará una velocidad de subida de datos alta, una velocidad de bajada alta, una oferta de canales básica con un target TV para toda la familia. Si el uso de internet es medio o bajo y el uso de televisión es elevado entonces le recomendará una velocidad de subida de datos baja, una velocidad de bajada alta, y una oferta de canales ampliada con un target TV adecuado a su perfil de visionado.

Una vez determinados los valores de las características de la tarifa más adecuada se puede identificar la tarifa concreta a partir de los valores concretos de consumo del usuario y los precios y características específicas de las tarifas ofrecidas de telefonía y de Internet+TV ofrecidas por el operador.

- a) (2,5 puntos) Diseña la ontología del dominio descrito, incluyendo todos los conceptos que aparecen en la descripción e identificando los atributos más relevantes. Lista que conceptos forman parte de los datos de entrada del problema y que conceptos forman parte de la solución. (Nota: tened en cuenta que la ontología puede necesitar modificaciones para adaptarla al apartado siguiente, de forma que todos los predicados hagan relacion a conceptos y atributos de la ontología).

En este problema la ontología ha de incorporar, como mínimo, todos los datos que se introducen en el formulario del usuario en el momento del registro al portal. Para dar soporte a las reglas del apartado b también incluiremos como atributos las características abstractas de usuario y las características abstractas de tarifas.



En este caso todas las clases del diagrama están asociadas a los datos de entrada (para modelar el uso que el usuario ha hecho de esos servicios y las tarifas que ha tenido en el pasado). La solución

está asociada a todas las clases excepto **Usuario** y **Destinatario**, ya que al recomendar tarifas éstas tienen asociadas los servicios que las componen.

De la ontología resultante cabe remarcar que si intentáramos hacer una ontología expresiva del dominio tenderemos a añadir muchos conceptos como los servicios ofrecidos por el operador (siendo entonces Telefonía, Mensajería, ... Internet Cable subclases de Servicio), los mensajes individuales (SMS y MMS) enviados y su destinatario, y todas y cada una de las conexiones a Internet realizadas por el usuario (http, https, smtp, ftp, etc...), pero al no hacernos falta para este problema solo hemos añadido sumatorios, medias y otras características al uso de mensajería e internet fijo y móvil. También se podría haber creado la clase Perfil Usuario para albergar toda la información derivada del análisis de la actividad del usuario, pero como este problema casi no indica atributos para la clase Usuario se ha decidido no crear una clase aparte para su perfil. Lo que si se ha hecho es modelar bien:

- la taxonomía de tarifas ofrecidas por la compañía y los servicios que las componen,
- las relaciones del usuario con las tarifas que contrata/ha contratado/se le recomiendan y los servicios que el usuario usa/consume.

Existen varios conceptos que comparten atributos y/o relaciones, y que esto ha sido algo que se ha tenido muy en cuenta a la hora de crear super-clases con las características comunes, o para decidir entre crear subclases o solo añadir un atributo **tipo**. La regla general es que hemos creado subclases cuando una o varias tienen atributos diferentes y/o relaciones diferentes. Ese es el caso de las subclases de **Programa** (algunas subclases tienen atributos diferentes) o de **Tarifa** (todas las subclases se relacionan con clases diferentes, y muchas tienen atributos diferentes). Se podría haber optado por no crear las clases **Mensajería** e **Internet Móvil** y haber incluido sus atributos dentro de la clase **Tarifa Móvil**, pero se han añadido ambas clases siguiendo el heurístico del diseño de ontologías que recomienda que ramas hermanas de la taxonomía (en este caso los servicios asociados a cada subclase de tarifas) tengan un nivel de granularidad similar. Se ha optado por no crear la clase **Persona** como superclase de **Usuario** y **Destinatario** ya que estas dos clases no comparten ningún atributo común, pero no sería incorrecto el añadir esa superclase.

Atributos:

- **Usuario:** *id_usuario* (string), *fidelidad* (enumeración: {baja, alta}), *uso_telefonia* (enumeración: {bajo, moderado, alto, intensivo}), *destino_llamadas* (enumeración: {al_operador, varios_operadores}), *uso_mensajería* (enumeración: {bajo, medio, alto}), *uso_internet_movil* (enumeración: {nulo, bajo, medio, intensivo}), *uso_internet_fijo* (enumeración: {nulo, bajo, medio, intensivo}), *uso_televisión* (enumeración: {nulo, ocasional, normal, elevado}), *perfil_visionado* (enumeración: {ninguno, variado, may_informativos, may_cultural, may_ficción, may_infantil, may_corazón})
- **Operador_Telecomunicaciones:** *nombre* (string)
- **Tarifa:** *nombre_tarifa* (string), *precio_mensual* (real positivo),
- **Tarifa_Movil:** *precio_consumo_mínimo* (real positivo), *precio_establecimiento_llamada* (real positivo), *num_minutos_llamada_incluidos* (entero), *velocidad_conexión_datos* (real), *precio_SMS* (real positivo), *precio_MMS* (real positivo), *meses_permanencia* (entero), *cuota_mensual_movil* (enumeración: {ninguna, baja, media, alta}), *consumo_mínimo* (enumeración: {no, bajo, medio, alto}), *coste_establecimiento_llamada* (enumeración: {ninguno, bajo, medio, alto}), *minutos_llamadas_incluidas* (enumeración: {medio, alto, muy_alto}), *velocidad_Internet* (enumeración: {no_tiene, 3G, 4G, 5G}), *GB_datos_incluidos* (enumeración: {ninguno, <2GB, 3..4GB, >4GB}), *coste_mensaje* (enumeración: {bajo, medio, alto}), *exigencia_permanencia* (enumeración: {alta, media, baja})
- **Tarifa_Cable:** *cuota_mensual_cable* (enumeración: {baja, media, alta}), *vel_bajada_datos_cable* (enumeración: {básica, media, alta}), *vel_subida_datos_cable* (enumeración: {básica, media, alta}), *oferta_canales* (enumeración: {básica, ampliada, máxima}), *target_TV* (enumeración: {infantil, juvenil, familiar, informativo, cultural, deportes, corazón})
- **Telefonía:** *num_telemófono* (entero), *num_llamadas_mañana* (entero), *num_llamadas_tarde* (entero), *num_llamadas_noche* (entero), *num_llamadas_laborables* (entero), *num_llamadas_festivos* (entero), *num_operadores_diferentes_llamadas* (entero), *duración_llamadas_mañana* (entero), *duración_llamadas_tarde* (entero), *duración_llamadas_noche* (entero), *duración_llamadas_laborables* (entero), *duración_llamadas_festivos* (entero)

- **Llamada:** fecha (fecha), hora_inicio (hora), minutos_duración (entero).
- **Destinatario:** num_teléfono (entero)
- **Mensajería:** media_SMS_mes (real), media_MMS_mes (real)
- **Internet_Movil:** GB_datos_mes (real positivo)
- **Internet_Cable:** meses_contrato (entero), media_GB_datos_bajada_mes (real positivo), media_GB_datos_subida_mes (real positivo), IP_fija (bool), modem_wifi (bool), firewall (bool), asistencia_telefonica (bool)
- **Canal:** nombre_canal (string), num_canal (entero)
- **Programa:** nombre_prog (string), duración_media_min (entero), año_1a_emisión(año), clasificación_edad (enumeración: {apto, +3, +7, +12, +18}), pay_per_view (bool), num_min_vistos (entero)
- **Serie_Ficción:** temporada (entero), episodio (entero)
- **Película:** director (string), género (enumeración: {cine_clásico, animación, ciencia_ficción, comedia, drama, acción, ...})

Como se puede ver, hemos decidido también representar las características abstractas como atributos (son los que tienen el nombre en *cursiva*). De esta manera todos los conceptos que aparecerán en las reglas están soportados por la ontología.

- b) (2,5 puntos) El problema descrito es un problema de análisis. Explica cómo lo resolverías usando clasificación heurística, identificando las fases necesarias y listando los predicados que se usan para ir de una fase a la siguiente. Da al menos 3 ejemplos de reglas para cada una de las fases de esta metodología, usando los conceptos de la ontología desarrollada en el apartado anterior. Inventa algún ejemplo concreto de tarifa si te hace falta.

En este caso el enunciado no solo nos propone un conjunto finito de características abstractas para describir al usuario, sino que además nos propone un conjunto de características abstractas para describir las tarifas. Dado que el problema a resolver es recomendar una tarifa concreta a un usuario, las características abstractas del usuario son la descripción del problema abstracto, y las características abstractas de tarifas son la descripción de la solución abstracta. A partir de este planteamiento lo que nos queda es describir las fases y elementos necesarios para aplicar la clasificación heurística al problema.

El primer elemento son los problemas concretos que hay que tratar. En este dominio los problemas concretos están definidos por los datos concretos del cliente que recibe el portal a través del formulario de registro (numero de llamadas, media de consumo de Internet, ...).

El segundo elemento son los problemas abstractos, que describen el problema mediante las características abstractas del usuario mencionadas en el enunciado (fidelidad, uso telefonía, ...) que hemos listado como atributos abstractos de la clase Usuario en el apartado anterior.

Para conectar los problemas concretos con los abstractos necesitamos definir las reglas de abstracción de datos, por ejemplo:

- si existe (Usuario.Tarifa_Actual.fecha_inicio) < jun 2016
or cardinalidad(Usuario.tarifa_anterior) > 2 entonces Usuario.fidelidad=alta;
- si Telefonía.num_operadores_diferentes_llamadas > 1
entonces Usuario.destino_llamadas=varios_operadores;
- si Internet_Movil.GB_datos_mes > 4GB entonces Usuario.uso_internet_movil=intensivo;
- si Internet_Movil.GB_datos_mes < 0.05GB entonces Usuario.uso_internet_movil=nulo;
- si Internet_Cable.media_GB_datos_bajada_mes > 30GB
or Internet_Cable.media_GB_datos_subida_mes > 10GB
entonces Usuario.uso_internet_fijo=nulo;
- si Media_Mensual(Usuario.ha_visto.num_min_vistos)>3600 minutos
entonces Usuario.uso_televisión=elevado;
- si cardinalidad(Usuario.ha_visto es_un Serie_Ficción) > 0.6* cardinalidad(Usuario.ha_visto)
entonces Usuario.perfil_visionado=may_ficción;
- si Mensajería.media_SMS_mes < 10 or Mensajería.media_MMS_mes < 5
entonces Usuario.uso_mensajería=bajo;

En las reglas anteriores el numero de tarifas anteriores se obtiene midiendo la cardinalidad de la

relación `tarifa_anterior` entre `Usuario` y `Tarifa`. Se ha supuesto la existencia de funciones (como `Media_Mensual`) para calcular medias, mediana, máximos, mínimos, etc.

El tercer elemento son las soluciones abstractas. En este caso el enunciado nos da un listado completo de características abstractas que permiten caracterizar las tarifas de móvil y de cable (exigencia de permanencia, cuota mensual móvil, oferta de canales...) identificadas como atributos abstractos de `Tarifa_Movil` y `Tarifa_Cable` en el apartado anterior.

Ejemplos de reglas de asociación heurística que ligán los problemas abstractos con las soluciones abstractas serían las siguientes (todas se mencionan explícitamente en el enunciado):

- si `Usuario.fidelidad=baja` y `Usuario.uso_teléfono=alto`
entonces `Tarifa_Movil.consumo_mínimo=alto` y `cuota_mensual=media`
- si `Usuario.fidelidad=alta` y `Usuario.uso_teléfono=bajo`
entonces `Tarifa_Movil.cuota_mensual=baja` y `exigencia_permanencia = alta`
- si `Usuario.uso_teléfono=intensivo`
entonces `Tarifa_Movil.coste_establecimiento_llamada=bajo`
y `Tarifa_Movil.consumo_mínimo=no`
y `Tarifa_Movil.minutos_llamadas_incluidas=(alto o muy_alto)`
- si `Usuario.uso_internet_fijo=intensivo` y `Usuario.uso_televisión=nulo`
entonces `Tarifa_Cable.vel_subida_datos_cable=alta`
y `Tarifa_Cable.vel_bajada_datos_cable=alta`
y `Tarifa_Cable.oferta_canales=básica` y `Tarifa_Cable.target_TV=familiar`
- si `Usuario.uso_internet_fijo=(medio o bajo)` y `Usuario.uso_televisión=elevado`
y `Usuario.perfil_visionado=may_cultural`
entonces `Tarifa_Cable.vel_subida_datos_cable=baja`
y `Tarifa_Cable.vel_bajada_datos_cable=alta`
y `Tarifa_Cable.oferta_canales=ampliada` y `Tarifa_Cable.target_TV=cultural`

El cuarto elemento son las soluciones concretas. En este caso son tarifas concretas, con todos sus detalles, entre las que se escoge una. Se puede optar por hacer reglas que mapeen directamente un conjunto de características abstractas a tarifas concretas, pero esto es poco realista ya que serían reglas difíciles de mantener y que requerirían cambios continuamente. Una alternativa es que en el refinamiento escojamos valores concretos de los parámetros de la tarifa que buscamos, y con esos valores se montará una query a la base de datos de tarifas que tiene el portal. Algunos ejemplos de reglas siguiendo esta idea serían las siguientes:

- si `Tarifa_Movil.cuota_mensual_movil=baja` y `Tarifa_Movil.consumo_mínimo=bajo`
entonces `addQuery(Tarifa_Movil.precio_mensual < 2 euros y`
`Tarifa_Movil.precio_consumo_minimo < Usuario.tarifa_actual.precio_consumo_minimo)`
- si `Tarifa_Movil.exigencia_permanencia=baja` y `Tarifa_Movil.cuota_mensual_movil=media`
entonces `addQuery(Tarifa_Movil.precio_mensual < 5 euros`
y `Tarifa_Movil.meses_permanencia < 6` y `Tarifa_Movil.precio_establecimiento_llamada ≤`
`Usuario.tarifa_actual.precio_establecimiento_llamada)`
- si `Tarifa_Movil.exigencia_permanencia=baja` y `Tarifa_Movil.cuota_mensual_movil=alta`
entonces `addQuery(Tarifa_Movil.precio_mensual < 9 euros`
y `Tarifa_Movil.meses_permanencia = 0` y `Tarifa_Movil.velocidad_conexion_datos ≥`
`Usuario.tarifa_actual.velocidad_conexion_datos)`
- si `Tarifa_Movil.cuota_mensual_movil=ninguna` y `Tarifa_Cable.cuota_mensual_cable=alta`
y `Tarifa_Cable.vel_bajada_datos_cable=alta` y `Tarifa_Cable.oferta_canales=ampliada`
y `Tarifa_Cable.target_TV=deportes`
entonces `addQuery(Tarifa_Cable.precio_mensual < Usuario.tarifa_actual[Cable].precio_mensual)`
y `add_Pack("FullSportTV")`
- si `Tarifa_Movil.minutos_llamadas_incluidas=muy_alto`
y `Tarifa_Movil.cuota_mensual_movil=alta`
y `Tarifa_Movil.GB_datos_incluidos= >4GB` y `Tarifa_Cable.cuota_mensual_cable=alta`
y `Tarifa_Cable.vel_bajada_datos_cable=alta` y `Tarifa_Cable.oferta_canales=básica`
entonces `addQuery(Tarifa_Movil+Cable.precio_mensual <`
`(Usuario.tarifa_actual[Movil].precio_mensual + Usuario.tarifa_actual[Cable].precio_mensual)`
y `Tarifa_Movil+Cable.meses_permanencia = 0 )`

Es importante remarcar que en estas reglas de refinamiento estamos incorporando datos del problema concreto (e.g., la tarifa actual del usuario) en la query para afinar la búsqueda de tarifas concretas.

2. (5 puntos) El juego Minecraft se está convirtiendo en un benchmark para la evaluación de agentes inteligentes en entornos virtuales, dadas sus necesidades de planificación a largo plazo junto a control motor de inputs complejos. Queremos enfocarnos en resolver la primera de estas tareas, y en concreto la adquisición de recursos y herramientas.

La creación de objetos requiere de tareas de planificación complejas para la adquisición y procesado de objetos. Para ello, buscamos generar un planificador que, teniendo en cuenta el terreno, recursos naturales y estaciones de procesado sea capaz de fabricar objetos a demanda. Para esta primera versión, vamos a ignorar varias complejidades, como que el jugador tenga más o menos objetos de un mismo tipo (por ejemplo, el jugador puede tener o no tener palos), o que haya un número máximo de objetos que se pueden llevar.

Un planificador recibe tanto las peticiones de objetos a obtener como los diferentes lugares conocidos y un mapa de accesibilidad (por ejemplo, la casa está conectada al bosque y a las minas, pero para ir al desierto desde la casa es necesario pasar por el bosque). Los lugares tienen tanto recursos naturales disponibles (por ejemplo piedra o madera) como estaciones de procesado (por ejemplo hornos o mesas de trabajo). El planificador también recibe las recetas de procesado para fabricar nuevos objetos y el inventario y posición del jugador al principio del problema.

El jugador puede desplazarse entre los diferentes lugares en función del mapa de accesibilidad. También, si dispone de un objeto que es una estación de procesado, puede colocarlo en el lugar donde está de forma que la deja disponible para su uso, pero deja de tener el objeto en sí.

Para recolectar recursos naturales, se requiere que el jugador esté en un lugar que disponga de ellos. Además, opcionalmente habrá recursos que requieran que el jugador disponga de la herramienta adecuada para recolectarlos. En caso de requerirse, cada recurso tiene un conjunto de posibles herramientas que se podrían usar (por ejemplo, tanto un pico *de madera* o uno *de piedra etc* pueden coger piedra, pero solo un pico de piedra puede coger hierro).

Para fabricar un objeto, se requiere que el jugador disponga de objetos concretos, de 1 hasta 9, dependiendo de la receta. También es posible que la receta requiera de hacerse en una estación de procesado, que debe estar presente (y colocada) en el lugar actual. Dos recetas diferentes pueden requerir de los mismos objetos y estaciones de procesado, y dar lugar a dos objetos diferentes (por ejemplo, con tablas de madera, sin estación de procesado, pueden fabricar una mesa de trabajo o palos). Para simplificar el manejo de objetos, asumimos que fabricar el objeto no elimina los ingredientes del inventario porque el jugador siempre recoge y fabrica muchos más objetos de los necesarios.

Queremos un planificador que modele todos los pasos del jugador para conseguir los objetos en el mínimo de acciones posibles, y detallando cada vez que se realizan las cuatro acciones detalladas (desplazarse, colocar estación, recolectar y fabricar). Por ejemplo, un plan posible para una tarea sencilla de hacer un pico de piedra (empezando en la casa y con el inventario vacío) sería: ir al bosque, recoger madera, ir a casa, fabricar tablas de madera, fabricar palos de madera, fabricar un pico de madera en la mesa de trabajo, bajar a la mina, coger piedra con el pico de madera, fabricar una mesa de trabajo, colocar la mesa de trabajo y finalmente fabricar un pico de piedra en la mesa de trabajo.

- a) Describe el dominio (incluyendo predicados, acciones, etc...) usando PDDL. Ten en cuenta que el modelo del dominio ha de poderse extender a más o menos recetas, con más o menos ingredientes, nuevos lugares y recursos naturales.

Existen formas muy variadas de modelar este dominio en PDDL. La solución adjunta es un ejemplo. Además de lo habitualmente comentado en las rúbricas de estos exámenes (no usar forall de forma innecesaria, minimizar operadores, etc.) listaremos errores bastante comunes, así como alternativas igual de correctas a la solución propuesta:

- (-0.5) No permitir más de una forma de fabricar un ítem (objeto), y/o que una combinación de ingredientes y estación genere diferentes productos. Esto se hace de forma implícita en muchos modelos, pero en otros se convierte en un problema.
- (-0.25) Incluir un fluente contador de acciones para optimizar. Los planificadores de base ya optimizan en número de acciones, y esto añade una complejidad innecesaria.

- (-0.5) Tener requisitos de estación/herramienta en la cabecera de las acciones **es aceptable** y óptimo, pero solo si se permite tener 'no estación' o 'no herramienta' como objeto en el fichero de problema. En ausencia de éstos, cualquier plan que requiera tener herramientas/estaciones en las acciones no permite fabricar ningún objeto que no los tenga.
- (-0.25) Usar *exists* o *forall* de forma innecesaria. No se ha penalizado en casos donde el uso de *exists/forall* no implique mucha pérdida de eficiencia, pero sí en los casos donde escala muy mal.
- Dado el número fijo de ítems máximos por receta, la relación entre ingredientes y recetas se puede hacer mediante (hasta) 9 predicados iterados con un *forall* (ejemplo) o un predicado con 10 elementos (véase: (ingredientes ?receta -receta ?i1 ... ?i9 -item)). Ambos son igual de válidos (pero el segundo requiere tener del ítem 'vacío' o 'nada'). Mientras que un *forall* es ineficiente, también es ineficiente hacer un match de 9 objetos en un predicado, así que es cuestión de gustos (y queda fuera del nivel de la asignatura saber cuál es mejor realmente). Sería mejor el caso de *forall*, especialmente si hay muchas recetas que usan menos de 9 ingredientes, ya que de media requiere hacer match de menos objetos (a pesar del coste añadido del *forall*).
- La relación de accesibilidad entre estados puede hacerse uni- o bidireccional. Ya que probablemente se usa *adl* como requisito, sería mejor hacerla unidireccional para ahorrar memoria, pero no se penaliza hacerlo de forma contraria.
- Para mejorar la eficiencia, la solución presenta algunos predicados filtro, cortando las acciones que generan recursos mirando que no se tengan ya. No se ha descontado por no tenerlos (ya que es más fiel al paradigma declarativo), pero como ejercicio didáctico enseñamos cómo se añadirían. Su uso en estas acciones, dado que comprueban muchas cosas, acelera notablemente el programa en problemas grandes. En acciones como colocar o desplazarse, por el contrario, añadirían un overhead (cuestan de comprobar y no eliminan mucha carga de cómputo al quitar el resto de la precondition).
- Utilizamos un predicado *es_vacio* en la acción de fabricar, para referenciar la 'no-estación'. Si queremos poder 'fabricar' sin estación, requeriríamos de tener una estación 'no-estación' colocada en todos los lugares. Con este predicado, podemos saber si la estación es 'vacío' y saltar esa comprobación.

Una solución equilibrada sería la siguiente:

```
(define (domain Minecraft)
  (:requirements :adl :typing)
  (:types lugar item receta - object
          herramienta estacion recurso - item)
)

(:predicates
  ; Predicados agrupados por la primera acción que los necesita por claridad:
  ; Movimiento:
  (en ?l -lugar)
  (accesible ?l1 ?l2 - lugar)
  ; Recolecta
  (recolectable_en ?r - recurso ?l - lugar)
  (en_inventario ?i - item)
  (se_coge_con ?r -recurso ?h -herramienta)
  ; Colocar Estaciones
  (colocada ?e - estacion ?l - lugar)
  ; Fabricación
  (produce ?r - receta ?producto - item)
  (requiere_estacion ?r - receta ?e - estacion)
  (requiere_ingr ?r - receta ?i - item)
  (es_vacio ?e - estacion)
)
```

```

(:action ir
  :parameters (?lo ?ld - lugar)
  :precondition (and (en ?lo) (or (accesible ?lo ?ld) (accesible ?ld ?lo)))
  :effect (and (not (en ?lo)) (en ?ld))
)

(:action recolectar
  :parameters (?l - lugar ?r - recurso ?h - herramienta)
  :precondition (and (en ?l) (not (en_inventario ?r)) (recolectable_en ?r ?l)
                    (se_coge_con ?r ?h)(en_inventario ?h))
  :effect (en_inventario ?r)
)

(:action colocar
  :parameters (?e - estacion ?l - lugar)
  :precondition (and (en_inventario ?e) (en ?l))
  :effect (and (not (en_inventario ?e)) (colocada ?e ?l))
)

(:action fabricar
  :parameters (?p - item ?r - receta ?l - lugar ?e - estacion)
  :precondition (and (en ?l) (not (en_inventario ?p))
                    (produce ?r ?p) (requiere_estacion ?r ?e)
                    (or (es_vacio ?e) (colocada ?e ?l))
                    (forall ?i -item) (imply (requiere_ingr ?r ?i)
                                              (en_inventario ?i)))
  :effect (en_inventario ?p)
)

```

- b) Pedimos una tarea para hacer un pico de diamante (típico del videojuego). El jugador comienza con piedra y palos, y está en la casa. La casa contiene un horno y una mesa de trabajo. El mapa es sencillo: el bosque conecta con la casa, la casa con la mina, y la mina con la mina profunda. En el bosque se puede recolectar madera, en la mina se puede recolectar piedra (con cualquier pico), carbón (cualquier pico) y mena de hierro (cualquier pico **menos** el pico de madera), y en la mina profunda se puede recolectar carbón, hierro y diamantes (con pico de hierro o diamante). A continuación se detallan las recetas relevantes:

producto	estación	ingredientes
Tablas	-	madera
Palos	-	Tablas
Mesa de trabajo (MDT)	-	tablas
Horno	MDT	piedra
Hierro	Horno	mena de hierro, carbón
Pico de Madera	MDT	palos, tablas
Pico de Piedra	MDT	palos, piedra
Pico de Hierro	MDT	palos, hierro
Pico de Diamante	MDT	palos, diamantes

Describe este problema usando PDDL. Da una breve explicación de cómo modelas el problema.

En este caso el modelado del problema está totalmente marcado por el modelado del dominio del apartado anterior. El resultado es el siguiente:


```

(define (problem PicoDiamante)
  (:domain Minecraft)
  (:objects Casa Bosque Mina MinaProf - lugar
            Tablas Palo - item
            Madera Piedra MenaHierro Carbon Diamante - recurso
            pMadera pPiedra pHierro pDiamante Mano - herramienta
            MDT Horno NoEstacion - estacion
            RTablas ... RpDiamante - receta
  )
  (:init
    ; Estado inicial
    (en Casa)
    (en_inventario Piedra)
    (en_inventario Palo)

    ; Mapa y recursos
    (accesible Casa Bosque)
    (accesible Casa Mina)
    (accesible Mina MinaProf)

    (recolectable_en Madera Bosque)
    (se_coge_con Madera Mano)
    (recolectable_en Piedra Mina)
    (se_coge_con Piedra pMadera)
    ...
    (se_coge_con Piedra pDiamante)
    ...
    (recolectable_en Diamante Minaprof)
    (se_coge_con Diamante pHierro)
    (se_coge_con Diamante pDiamante)

    ; Estaciones Colocadas
    (colocada MDT Casa)
    (colocada Horno Casa)

    ; Fabricación
    (produce RTablas Tablas)
    (requiere_estacion RTablas NoEstacion)
    (requiere_ingr RTablas Madera)

    ...

    (produce RpDiamante pDiamante)
    (requiere_estacion RpDiamante MDT)
    (requiere_ingr RpDiamante Palo)
    (requiere_ingr RpDiamante Diamante)

    ; Misc
    (es_vacio NoEstacion)
  )

  (:goal (tiene pDiamante))
)

```